

Chapter Two

Classes and Objects in Java

OOP Principles

Class is an OOP construct that consists of **data members** and **member functions**. Data members are nothing but simply variables that we declare inside the class so it is called **data member** of that particular class. Member functions are the function or you can say **method** which we declare inside the class so it is called member function of that particular class. The most important thing to understand about a class is that it defines a new **user defined/programmer defined data type**. Once defined, this new data type can be used to create **objects** of that type. Thus, a class is a template for an object, and an object is an **instance** of a class. Because an object is an instance of a class, you will often see the two words **object and instance** used interchangeably.

When we consider a Java program, it can be defined as a collection of objects that communicate via invoking each other's methods. Let us now briefly look into what do class, object, methods, and instance variables mean.

- ✓ **Object** - Objects have states and behaviors. Example: An object is an instance of a class.
- ✓ **Class** - A class can be defined as a template/blueprint that describes the behavior/state that the object of its type supports.
- ✓ **Methods** - A method is basically a behavior. A class can contain many methods. It is in methods where the logics are written, data is manipulated and all the actions are executed.
- ✓ **Instance Variables** - Each object has its unique set of instance variables. An object's state is created by the values assigned to these instance variables.

Syntax of class:

```
class classname
{
    type instance-variable1;
    type instance-variable2;
    //....
    type instance-variableN;
    type methodname1(parameter-list)
    {
        // body of method
    }
    type methodname2(parameter-list)
    {
        // body of method    }
    type methodnameN(parameter-list)
    {
        // body of method    } }
```

Instantiation

When you define a class, you declare its exact form and nature/template. You do this by specifying the data that it contains and the code that operates on that data. The data, or variables, defined within a class are called instance variables. The code is contained within methods.

```
public class MyPoint
{
    int x = 0;
    int y = 0;

    void displayPoint()
    {
        System.out.println("Printing the coordinates");
        System.out.println(x + " " + y);
    }
    public static void main(String args[])
    {
        MyPoint obj; // declaration
        obj = new MyPoint(); // allocation of memory to an object
        obj.x=10; //access data member using object.
        obj.y=20;
        obj.displayPoint(); // calling a member method
    }
}
Output :
Printing the coordinates
10 20
```

Here x and y are **data members** of class *MyPoint* and *displayPoint()* is a member function of the same class. In above program we can take as many objects as we want. From the above program one thing we should keep in mind is that when we want to access **data member** or **methods** of class we have to write **objectname.membername(dot operator)**

Syntax:

- accessing data member of the class: **objectname.datamember name;**
- accessing methods of the class: **objectname.method name();**

So for accessing data of the class: **we have to use (.) dot operator.**

NOTE: we can use or access data of any particular class without using (.) dot operator from inside that particular class only.

The program in above topic can easily show that how object is going to be declared and defined for any class.

Syntax of object:declaration

classnameobjectname; // declaration of object/refrnce

objectname = new classname(); //instantiating the object

//allocate memory to object (define object).or we can directly define object like this

classnameobjectname = new classname();

Chapter 2 Class and objects in Java

Now let us look one class which has two different object.

```
class Box
{
    double width;
    double height;
    double depth;
}
class BoxDemo2
{
    public static void main(String args[])
    {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;
        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;
        /* assign different values to mybox2's instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;
        // compute volume of first box
        vol = mybox1.width * mybox1.height * mybox1.depth;
        System.out.println("Volume is " + vol);
        // compute volume of second box
        vol = mybox2.width * mybox2.height * mybox2.depth;
        System.out.println("Volume is " + vol);
    }
}
```

Output :

Volume is 3000.0

Volume is 162.0

From the above program we can understand that each object has its own copies of the instance variables. This means that if you have two **Box** objects, each has its own copy of **depth**, **width**, and **height**. It is important to understand that changes to the instance variables of one object have no effect on the instance variables of another.

Assigning Object Reference Variables :

Suppose

Box b1 = new Box();

Box b2 = b1;

Here b1 is the object of class Box. And we assign b1 to b2 by b2=b1. Here we did not use new keyword for b2 so b1 and b2 will both refer to the same object. The assignment of b1 to b2 did

not allocate any memory or copy any part of the original object. It simply makes b2 refer to the same object as does b1. Thus, any changes made to the object through b2 will affect the object to which b1 is referring, since they are the same object.

NOTE: When you assign one object reference variable to another object reference variable, you are not creating a copy of the object, you are only making a copy of the reference. As we all know that, classes usually consist of two things instance variables and methods.

Here we are going to explain some fundamentals about methods. So we can begin to add methods to our classes

Class Methods

Methods are defined in a class body using the components as below

- ✓ *Return type*
- ✓ *Name of the method*
- ✓ *A list of parameters*
- ✓ *Body of the method.*

Syntax:

return type method name (list of parameters)

```
{  
    Body of the method  
}
```

return type specifies the type of data returned by the method after manipulation. This can be any valid data type including class types that you create. If the method does not return a value, its return type must be **void**. Means you can say that void means no return. Methods that have a return type other than **void** return a value to the calling routine using the following form of the **return** statement:

return value; Here, value is the value returned.

The **method name** is any legal identifier.

The **list of parameter** is a sequence of type and identifier pairs separated by commas. Parameters are essentially variables that receive the value of the arguments passed to the method when it is called. If the method has no parameters, then the parameter list will be empty.

Let us look at one example of class which have methods and data members both.

```
import java.util.Scanner;  
class Box  
{  
    double width;  
    double height;  
    double depth;  
    void volume()    // display volume of a box  
    {  
        System.out.print("Volume is : ");  
        System.out.println(width * height * depth);  
    }  
}
```

```
    }  
}  
class BoxDemo  
{  
    public static void main(String args[])  
    {  
        Box box1 = new Box(); // defining object box1 of class Box  
        Scanner s = new Scanner(System.in);  
        System.out.print("Enter Box Width : ");  
        box1.width = s.nextDouble();  
        System.out.print("Enter Box Height : ");  
        box1.height = s.nextDouble();  
        System.out.print("Enter Box Depth : ");  
        box1.depth = s.nextDouble();  
        // display volume of box1  
        box1.volume(); // calling the method volume  
    }  
}
```

Output:

```
Enter Box Width : 15  
Enter Box Height : 20  
Enter Box Depth : 10  
Volume is : 3000.00
```

Here width, height and depth are data members of class Box and **void volume()** is method of class Box. Here method has no parameter and no return value.

Now let us look at same program but in different way.

```
import java.util.Scanner;  
class Box  
{  
    double width;  
    double height;  
    double depth;  
    double volume()  
    {  
        return width * height * depth;  
    }  
}  
class BoxDemo  
{  
    public static void main(String args[])  
    {
```

```
double vol;  
Box box1 = new Box(); // defining object box1 of class Box  
Scanner s = new Scanner(System.in);  
System.out.print("Enter Box Width : ");  
box1.width = s.nextDouble();  
System.out.print("Enter Box Height : ");  
box1.height = s.nextDouble();  
System.out.print("Enter Box Depth : ");  
box1.depth = s.nextDouble();  
// display volume of box1  
vol = box1.volume(); // calling the method volume  
System.out.println("Volume is : " +vol);  
}  
}
```

Output:

```
Enter Box Width : 15  
Enter Box Height : 20  
Enter Box Depth : 10  
Volume is : 3000.00
```

Here in above program volume() method has return type double so we took one vol variable. It is a local variable of class BoxDemo and it catch the returned value by volume method of class Box. One thing must keep in mind that the data type of vol and return type of volume() method always be same.

```
import java.util.Scanner;  
class Box  
{  
    double width;  
    double height;  
    double depth;  
    double volume(double w, double h, double d)  
    {  
        return width * height * depth;  
    }  
}  
class BoxDemo  
{  
    public static void main(String args[])  
    {  
        double vo,wth,ht,dth;  
        Box box1 = new Box(); // defining object box1 of class Box  
        Scanner s = new Scanner(System.in);  
        System.out.print("Enter Box Width : ");  
        wth = s.nextDouble();
```

```
System.out.print("Enter Box Height : ");  
ht = s.nextDouble();  
System.out.print("Enter Box Depth : ");  
dth = s.nextDouble();  
// display volume of box1  
vol = box1.volume(wth,ht,dth); // calling the method volume  
System.out.println("Volume is : " +vol);  
    }}
```

Output:

```
Enter Box Width : 15  
Enter Box Height : 20  
Enter Box Depth : 10  
Volume is : 3000.00
```

Here in above program **volume()** method has three parameters as well as double return type. Here we took three extra local variables in class BoxDemo and pass them in function calling. One thing must keep in mind that in defining and calling of method, the sequence of data type of parameter must be same in both. Java supports a special type of methods, called constructor that enables an object to initialize itself when it is created.

Constructors

Constructors are special member methods that have the same name as the class itself. Constructors do not have specific return type and not even void. This is because they return the instance of the class itself. A constructor is automatically called when an object is created.

Syntax:

```
Constructor_name([arguments])  
    {  
        // body  
    }
```

Constructors are generally of two types.

1. Non-Parameterized
2. Parameterized

1. Non-Parameterized:

```
// Non - parameterised constructor / default constructors  
class Point1  
{  
    int x;  
    int y;  
    Point1() //constructor of class  
    {  
        x = 10;  
        y = 20;  
    }  
}
```

```
void display()
{
    System.out.println("\n\n\t----Printing the coordinates----");
    System.out.println("\t\t\t" + x + " " + y);
}
}
class pointDemo
{
    public static void main(String args[])
    {
        Point1 p1 = new Point1(); // constructor will be call automatically from
here
        p1.display();
    }
}
Output:
-----Printing the coordinates-----
10 20
```

2. Parameterized:

```
// parameterised constructor
import java.util.Scanner;
class Point2
{
    int x;
    int y;
    Point2(int a, int b)
    {
        x = a;
        y = b;
    }
    void display()
    {
        System.out.println("\n\n\t----Printing the coordinates----");
        System.out.println("\t\t\t" + x + " " + y);
    }
}
class pointDemo
{
    public static void main(String args[])
    {
        inti,k;
```

```
Scanner s = new Scanner(System.in);
System.out.print("Enter int value for i : ");
i = s.nextInt();
System.out.print("Enter int value for k : ");
k = s.nextInt();
Point2 p1 = new Point2(i,k);
p1.display();
    }
}
```

Output:

```
Enter int value for i : 10
Enter int value for k : 20
-----Printing the coordinates-----
    10  20
```

We have already seen several keywords so far. But here we are going to learn few regularly used keywords. **new**, **this**, **static**, **super**, **final**

The **super** and **final** keywords will demonstrate in further chapter.

new:

The new keyword dynamically allocates memory for an object.

Syntax:

class_name object_name = new class_name();

EX.

```
Box b1 = new Box();
```

```
Box b2 = new Box();
```

We have already seen so many programs in which we used **new** keyword for creating objects.

this:

This keyword is the name of a reference that refers to a calling object itself. One common use of the **this** keyword is to reference a class hidden data fields. You can have local variables, including formal parameters to methods which overlap with the names of the class` instance variables. The local variable hides the instance variable so we use **this** keyword. Another common use of this keyword is to enable a constructor to invoke another constructor of the same class. Java requires that the this(arg-list) statement appear first in the constructor before any other statements.

EX :

```
public class This_demo
{
    public static void main(String[] args)
    {
        abc a1 = new abc();    //call non parameterize constructor
    }
}
class abc
{
```

```
int x,y;  
abc()  
{  
    this(10,20); //this will call another constructor  
}  
abc(int x,int y)  
{  
    this.x=x+5; //it will set class` member x  
    this.y=y+5; //it will set class` member y  
    System.out.println("local method`s x = " +x);  
    System.out.println("local method`s y = " +y);  
    Print_data();  
}  
public void Print_data()  
{  
    System.out.println("x = " +x);  
    System.out.println("y = " +y);  
}  
}
```

Output :

```
local method`s x = 10  
local method`s y = 20  
x = 15  
y = 25
```

static :

A class member must be accessed with the use of an object of its class but sometimes we want to define a class member that will be used independently without creating any object of that class. It is possible in java to create a member that can be used by itself, without reference to a specific instance. To create such a member, precede its declaration with the keyword **static**. When a member is declared static, it can be accessed before any objects of its class are created, and without reference to any object. One can declare both methods and variables to be static. The most common example of a static member is main().main() is declared as static because it must be called before any object exist. Instance variables declared as static are actually, global variables. When objects of its class are declared, no copy of a static variable is made. Instead, all instances of the class share the same static variable.

Method declared as static have several restrictions:

1. Can call only other static methods.
2. Only access static data.
3. Cannot refer to **this**

One can also declare a static block which gets executed exactly once, when the class is first loaded.

EX :

```
public class Static_Demo
{
    public static void main(String[] args)
    {
        display(40); //call static method
    }
    static int i = 5;
    static int j;
    int k = 10;
    static void display(int a)
    {
        System.out.println("a = "+a); // here a = 40
        System.out.println("i = "+i); // here i = 5
        System.out.println("j = "+j);
        // here j = 50( because of static block j = i * 10 = 5 * 10 =50 )
        System.out.println("k = "+k);
        //can,t make a static reference to the non-static field k
    }
    static
    {
        System.out.println("Static block initialized.....");
        j = i * 10;
    }
}
```

Output :

Static block initialized.....

a = 40

i = 5

j = 50

Encapsulation

One can control what parts of a program can access the member of a class. By controlling access, one can prevent misuse. For Example, allowing access to data only through a well-defined set of methods, one can prevent misuse of that data. Thus, when correctly implemented, a class creates “black box”. How a member can be accessed is determined by the access specifier that modifies its declaration. Java provides a set to access specifiers. Some aspects of access control are related to inheritance or packages.

Java’s access specifiers are:

Chapter 2 Class and objects in Java

public:

- ✓ When a member of a class is specified by the public specifier, then that member can be accessed by any other code.
- ✓ The public modifier makes a method or variable completely available to all classes.
- ✓ Also when the class is defined as public, it can be accessed by any other class.

private:

- ✓ To hide a method or variable from other classes, private modifier is used.
- ✓ A private variable can be used by methods in its own class but not by objects of any other class.
- ✓ Neither private variables nor private methods are inherited by subclass.
- ✓ The only place these variables and methods can be seen is from within their own class.

protected:

- ✓ protected applies only when inheritance is involved.
- ✓ If you want to allow an element to be seen outside your current package, but only to classes that are inherited from your class directly, then declare that element as protected.

default:

- ✓ We have seen that when no access control modifier is specified, it is called as default access.
- ✓ Classes written like this are not accessible in other package.
- ✓ Any variable declared without a modifier can be read or changed by any other class in the same package.
- ✓ Any method declared the same way can be called by any other class in the same package.
- ✓ When a member does not have an explicit access specification,
- ✓ it is visible to subclasses as well as to other classes in the same package.

The following table summarizes the levels of access control.

Access	Public	Protected	Default	Private
From the same class	Yes	Yes	Yes	Yes
From any class in the same package	Yes	Yes	Yes	No
From any class outside the package	Yes	No	No	No
From a sub - class in the same package	Yes	Yes	Yes	No
From a sub - class outside the same Package	Yes	Yes	Yes	No

Method overloading in java:

Method overloading:

A class can contain any number of methods. Methods can be with parameter and without parameter. The parameter in a method are called type signature. It is possible in java to define two or more methods within the same class that ***share the same name***, but ***with different parameter declarations (type signatures)***. When this is the case, the methods are said to be overloaded, and the process is referred to as method overloading. Overloading methods demonstrate the concept of **polymorphism**. When an overloaded method is invoked, java uses the type and/or number of arguments as its guide to determine which version of the overloaded method to call. Thus, overloaded methods must differ in the type and/or number of their parameters. Overloaded methods may have different return types. When java encounters a call to an overloaded method, it simply executes the version of the method whose parameters match the arguments used in the call.

EX :

```
public class MethodOver
{
    int n1;
    int n2;
    MethodOver()
    {
        n1 = 10;
        n2 = 20;
    }
    void square()
    {
        System.out.println("The Square is " + n1 * n2);
    }
    void square(int p1)
    {
        n1 = p1;
        System.out.println("The Square is " + n1 * n2);
    }
    void square(int p1, int p2)
    {
        n1 = p1;
        n2 = p2;
        System.out.println("The Square is " + n1 * n2);
    }
    public static void main(String args[])
    {
        MethodOver obj1 = new MethodOver();
        obj1.square(); //call non parameterise method
    }
}
```

```
        obj1.square(4); //call method which has 1 argument
        obj1.square(7,8); //call method which has 2 argument
    }
}
```

Output :

```
The Square is 200
The Square is 80
The Square is 56
```

You can see that here we have 3 square methods with different argument. It's called method overloading. Constructor overloading in java: Along with method overloading, we can also overload constructors. Constructors having the same name with different parameter list is called constructor overloading.

EX :

```
class Point
{
    int x;
    int y;
    Point(int a, int b)
    {
        x = a;
        y = b;
    }
}
class Circle
{
    int originX;
    int originY;
    int radius;
    //Default Constructor
    Circle()
    {
        originX = 5;
        originY = 5;
        radius = 3;
    }
    // Constructor initializing the coordinates of origin and the radius.
    Circle(int x1, int y1, int r)
    {
        originX = x1;
        originY = y1;
        radius = r;
    }
}
```

```
    }  
    Circle(Point p, int r)  
    {  
        originX = p.x;  
        originY = p.y;  
        radius = r;  
    }  
    void display()  
    {  
        System.out.println("Center at " + originX + " and " + originY);  
        System.out.println("Radius = " + radius);  
    }  
    public static void main(String args[])  
    {  
        Circle c1 = new Circle();  
        Circle c2 = new Circle(10,20,5);  
        Circle c3 = new Circle(new Point(15,25),10);  
        c1.display();  
        c2.display();  
        c3.display();  
    }  
}
```

Output :

```
--Center at 5 and 5  
Radius = 3  
--Center at 10 and 20  
Radius = 5  
--Center at 15 and 25  
Radius = 10
```

Above program is quite complicated here i am giving you perfect flow of program. First of all note one thing that **new ClassName()** this is a short syntax of creating object of any class. And we all know that when we create object the constructor of that class will be called automatically. So in our program first of all due to syntax `Circle c1 = new Circle();` non parameterize constructor will be called for object c1 so we get output like Center at 5 and 5 Radius = 3 in `c1.display()`. Next due to syntax `Circle c2 = new Circle(10,20,5);` constructor which has 3 arguments will be called for object c2 so we get output like Center at 10 and 20 Radius = 5 in `c2.display()`. Now when we define object c3 our syntax is like `Circle c3 = new Circle(new Point(15,25),10);` so first of all it will create object for Point class so constructor of point class will be called and it will set parameter x and y. Then constructor of circle class which has Point class object as an argument along with one int argument will be called and set all parameter as per program and we get output like Center at 15 and 25 Radius = 10 in `c3.display()`.

We can also write

```
Point p1 = new Point(15,25);
```

```
Circle c3 = new Circle(p1,10);
```

This is how we can pass object as an argument in constructor. We have already seen Call by reference in which we pass object as a method argument. Now in next topic we will discuss about how you can return an object.

INHERITANCE in Java

In OOP, in order to solve actual problems, you often organize classes in **hierarchy to avoid duplication and reduce redundancy** and this approach is called **Inheritance**. The idea behind **inheritance** in Java is that you can create **new classes** that are **built upon existing classes**. When you inherit from an existing class, you can **reuse methods and fields/data** of the parent/base class. Moreover, you can add new methods and fields in your current class also. The classes in the lower hierarchy inherit all the variables (static attributes) and methods (dynamic behaviors) from the higher hierarchies. A class in the lower hierarchy is called a **subclass (or derived, child, extended class)**. A class in the upper hierarchy is called a **superclass (or base, parent class)**. By pulling out all the common variables and methods into the super classes, and leave the specialized variables and methods in the subclasses, **redundancy** can be greatly reduced or eliminated as these common variables and methods do not need to be repeated in all the subclasses.

As way of reusing the functionality and data defined by another class called **super class**, for example assume you need to build a new brand BMW car, are you going to build it from scratch? Not a good start what you will be doing is that you reuse the functionality of an already existing BMW and add the new special attributes and behaviors that the new BMW will have. You use **extends** keyword to inherit from a class.

Note:-

- **Super class** is a general class that defines common attribute and methods of a set of related classes called **subclass**. Therefore sub classes inherit all the methods and attributes of the super class and add its own unique element.
- The main intentions of using inheritance are code reusing/reducing redundancy and reducing complexity.

The syntax of Java Inheritance

```
class Subclass-name extends Superclass-name  
{  
    //methods and fields
```

Chapter 2 Class and objects in Java

}

The **extends** keyword indicates that you are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.

Consider the following example:

```
public class Student
{
    private String studentId;
    private String firstName;
    private String lastName;
    private String section;
    private Course[] course;

    //setter and getter method implemented here.....

}
//here is the course data type
public class Course
{
    private String courseName;
    private String creditHrs;
    private String courseCode;
    //setter and getter method implemented here ..... }

//regular student class
public class RegularStudent extends Student
{
    private String grade12CompletedYear;
    private String highSchoolName;

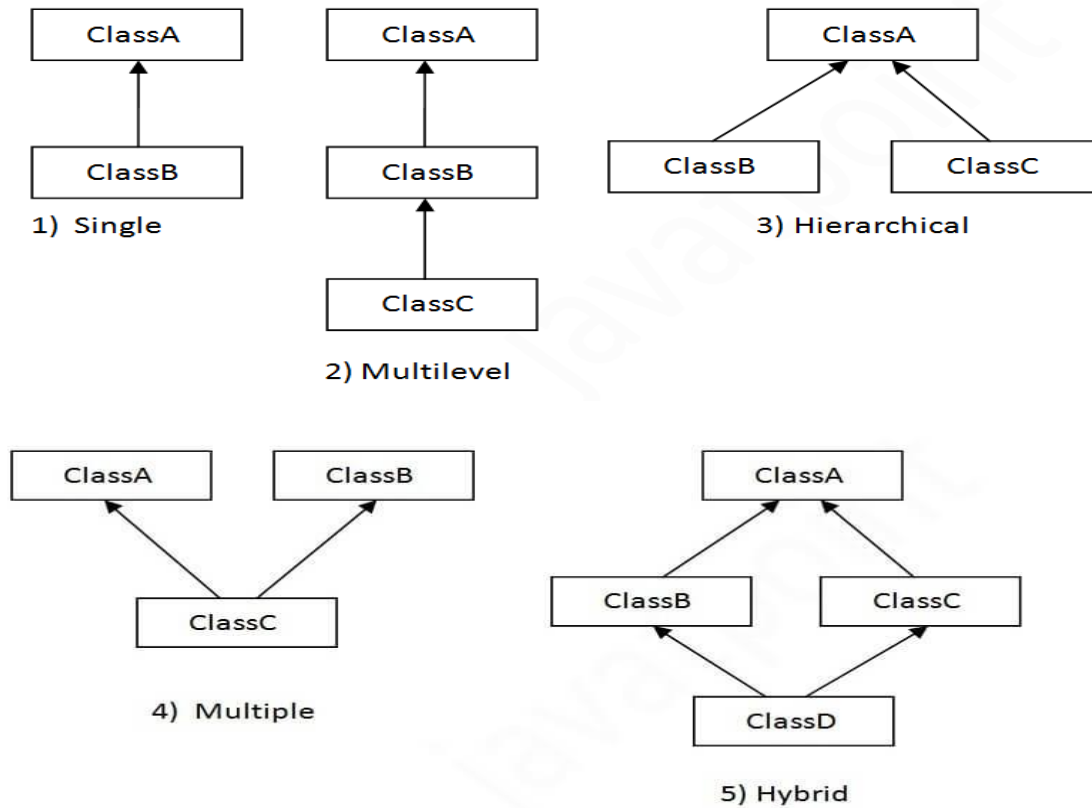
    public void acceptStudentInfo()
    {
        Scanner input=new Scanner(System.in);
        System.out.println("Enter studet id ");
        setStudentId(input.next());
        System.out.println("Enter first name ");
        setFirstName(input.next());
        System.out.println("Enter last name ");
        setLastName(input.next());
        System.out.println("Enter section ");
        setSection(input.next());
        System.out.println("Enter high school name ");
        setHighSchoolName(input.next());
        System.out.println("Enter grade 12 completed year");
```

```
setGradeCompleteYear(input.next());
Course[] course=new Course[2];
for(int i=0;i<course.length;i++)
{
    course[i]=new Course();
    System.out.println("Enter course name ");
    course[i].setCourseName(input.next());
    System.out.println("Enter course credit hour");
    course[i].setCreditHrs(input.next());
    System.out.println("Enter course code ");
    course[i].setCourseCode(input.next());
}
setCourse(course);
}
public void displayStudent()
{
    System.out.println("_____");
    System.out.println(this.getStudentId()+"    "+this.getFirstName()+"    "+this.getLastName()+"
"+this.getSection()+" "+this.getHighSchoolName()+" "+this.getGradeCompleteYear());
    System.out.println("_____");
    System.out.println("Course Name    Course Code    Credit Hour ");
    System.out.println("_____");
    for(int i=0;i<getCourse().length;i++)
    {
        System.out.print(this.getCourse()[i].getCourseName()+"
"+this.getCourse()[i].getCourseCode()+"    "+this.getCourse()[i].getCreditHrs());
        System.out.println();
    }
    System.out.println("_____");
}
```

As you can see the class **regular student** (subclass) inherits the entire attribute and the method of the **student** (super class). The **private members of the super class** can't be accessed directly by the subclass therefore subclass uses **setter** and **getter** method to access private member of the super class.

Types of inheritance in java

On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical. In java programming, multiple and hybrid inheritance is supported through interface only. You will learn about interfaces later in proceeding section.



Aggregation in Java

In java, you can also specify the relationship between multiple classes using the format of **HAS-A relationship**. This approach is called aggregation/composition. You can implement a composition by defining a reference of one class as a member variable of the other. In composition you do not need to use the keyword **extends**.

Example: composition of classes

```
class compA{  
    //member variables}  
class compB  
{ compA ob;  
}
```

Here in this example, the class **compB** can reuse all public members of the **compA** using its reference member **ob**.

The uses of super

Super is used for accessing a super class members, both variables and methods. You use super for reusing the super class constructor, which force you to make the first statement with in the sub class constructor. The second use of super is to refer to a super class member which is **redefined** in the **subclass**.

Example the first use of super

Add the following method to the class called student

```
public Student()
{
}
public Student(String studentId,String firstName,String lastName,String section,Course[]
course)// constructor
{
    this.studentId=studentId;
    this.firstName=firstName;
    this.lastName=lastName;
    this.section=section;
    this.course=course;
}
```

Add the following method to the class called Regular student

```
public RegularStudent()
{
}
public RegularStudent(String studentId,String firstName,String lastName,String section,
    Course[] course,String gradeCompleteYear,String highSchoolName)
{
    super(studentId, firstName, lastName, section, course);
    this.gradeCompleteYear=gradeCompleteYear;
    this.highSchoolName=highSchoolName;
}
```

Therefore, super is used to call the **super** class **constructor** without using getter and setter method we can access private members of the super class.

Chapter 2 Class and objects in Java

In the case of **multi-level inheritance** **super** refers to its **immediate** super class **constructor**.

Example the second use of super

Consider the following example

Class super_class

```
{
    int x;
    String name;
    Super_class (int x, String name)
    {
        this.x=x;
        this.name=name;
    }

    void display()
    {
        System.out.println("In the super "+x+" "+name);
    }
}
```

class sub_class extends super_class

```
{
    int y;
    int x;
    sub_class(int x,int y,String name)
    {
        super(x,name);
        this.y=y;
        this.x=x;
    }
    void display()
    {
        super.display();
        System.out.println("In the sub "+y);
        System.out.println("x In the super "+super.x);
        System.out.println("x In the sub "+x);
    }
}
```

class driver_inheritance

```
{
    Super_class sup1=new Super_class(1,"xxxxxx");
    Sub_class sub1= new sub_class(2,4,"yyyyyy");
    Sup1.display();
    Sub1.display();
}
```

Output of the program

```
In the super 1 xxxxxx
In the super 2 yyyyyy
In the sub 4
```

Consider the following example

```
public class Sup_class
{
    int x=100;
    public Sup_class()
    {
        System.out.println("Believe it or Not, Here you are at Super default constructor");
    }
    public Sup_class(int a)
    {
        System.out.println("Believe it or Not, Here you are at Super!" +a);
    }
}

public class Sub_class extends Sup_class
{
    int x=5;
    public Sub_class(int y)
    {
        System.out.println("Even if you tried you are the second " +y);
    }
    public void display()
    {
        System.out.println("X in the sub class " +x);
        System.out.println("X in the super class " +super.x);
    }
}

public class Main {
    public static void main(String[] args) {
        Sub_class sub=new Sub_class(90);
        sub.display();
    }
}
```

How constructors are invoked

Constructors are in an inheritance hierarchy are invoked from super class constructors to subclass constructor. If there is no explicit call to the super class constructor using super the empty constructor of

the super class will execute first. That is what makes the above program output different from the one you guess. The following is the output.

Output of the program

Even if you tried you are the second 90

X in the sub class 5

X in the super class100

Note:

A Super class Variable Can Reference a Subclass Object

A reference variable of a super class can be assigned a reference to any subclass derived from that super class. You will find this aspect of inheritance quite useful in a variety of Situations. It is important to understand that it is the type of the reference variable—not the type of the object that it refers to—that determines what members can be accessed. That is, when a reference to a subclass object is assigned to a super class reference variable, you will have access only to those parts of the object defined by the super class.

Protected Access Modifier

The **protected access modifier** is accessible within package and outside the package but through inheritance only. The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

Example of protected access modifier

The **A class** can be accessed from outside the class. But **msg** method of this class is declared as protected, so it can be accessed from outside the class only through inheritance.

```
public class A{
    protected void msg(){System.out.println("Hello");} //protected member
}

class B extends A{
    public static void main(String args[]){
        B obj = new B();
        obj.msg();
    } }

```

Output:Hello

Abstract class and Interface in java

Abstract class in Java

A class which is declared with the **abstract** keyword is known as an abstract class in Java. It can have abstract and non-abstract methods (method with the body). Before discussing the Java abstract class, let's understand the concept of abstraction in Java first.

Abstraction in Java

Abstraction is a process of hiding the implementation details and showing only functionality to the user. Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery. Abstraction lets you focus on what the object does instead of how it does it.

Ways to achieve Abstraction

There are two ways to achieve abstraction in java

1. Abstract class (0 to 100%)
2. Interface (100%)

Abstract class in Java

A class which is declared as abstract is known as an **abstract class**. It can have abstract and non-abstract methods. It needs to be extended and its method implemented.

Note:- It cannot be instantiated.

Points to Remember

An abstract class must be declared with an abstract keyword.

- ✓ It can have abstract and non-abstract methods.
- ✓ It cannot be instantiated.
- ✓ It can have constructors and static methods also.
- ✓ It can have final methods which will force the subclass not to change the body of the method.

Syntax:

```
abstract class A{  
}
```

Abstract Method in Java

A method which is declared as abstract and does not have implementation is known as an abstract method.

Example of abstract method

```
abstract void printStatus();//no method body and abstract
```

Example of Abstract class that has an abstract method

In this example, Bike is an abstract class that contains only one abstract method run. Its implementation is provided by the Honda class.

```
abstract class Bike{  
    abstract void run();  
}  
class Honda4 extends Bike{  
    void run(){System.out.println("running safely");}  
    public static void main(String args[]){  
        Bike obj = new Honda4();  
        obj.run(); }}  

```

Understanding the real scenario of Abstract class

In the following example, Shape is the abstract class, and its implementation is provided by **the Rectangle and Circle classes**. In this example, if you create the instance of Rectangle class, **draw()** method of **Rectangle class** will be invoked.

```
abstract class Shape{
    abstract void draw();
}
//In real scenario, implementation is provided by others i.e. unknown by end user
class Rectangle extends Shape{
    void draw(){System.out.println("drawing rectangle");}
}
class Circle1 extends Shape{
    void draw(){System.out.println("drawing circle");}
}
//In real scenario, method is called by programmer or user
class TestAbstraction1{
    public static void main(String args[]){
        Shape s=new Circle1();//In a real scenario, object is provided through method, e.g., get
        Shape() method
        s.draw(); }}

```

Interface in Java

An interface in Java is a blueprint of a class. It has static constants and abstract methods. The interface in Java is a mechanism to achieve abstraction. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritances in Java. In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method body.

**Java Interface also represents the IS-A relationship.
It cannot be instantiated just like the abstract class.**

Why we use Java interface?

There are mainly three reasons to use interface. They are given below.

- ✓ It is used to achieve abstraction.
- ✓ By interface, we can support the functionality of multiple inheritances.
- ✓ It can be used to achieve loose coupling.

How to declare an interface?

An interface is declared by using the **interface** keyword. It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default. A class that implements an interface must implement all the methods declared in the interface.

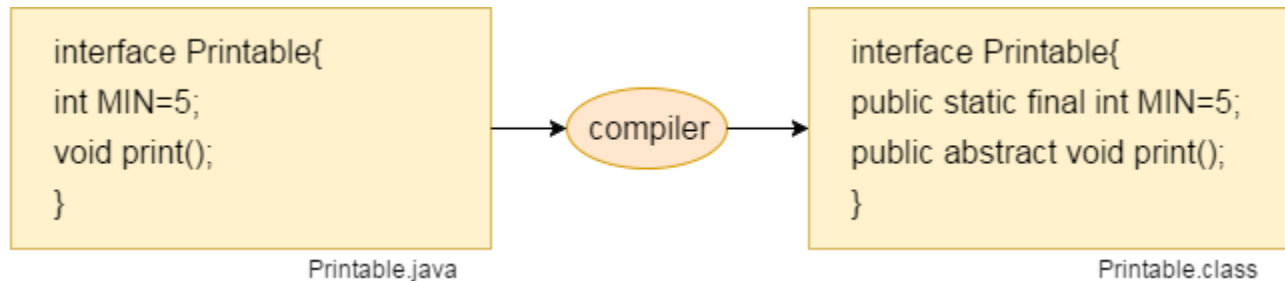
Syntax:

```
interface <interface_name>{  
  
    // declare constant fields  
    // declare methods that abstract  
    // by default.  
}
```

Internal addition by the compiler

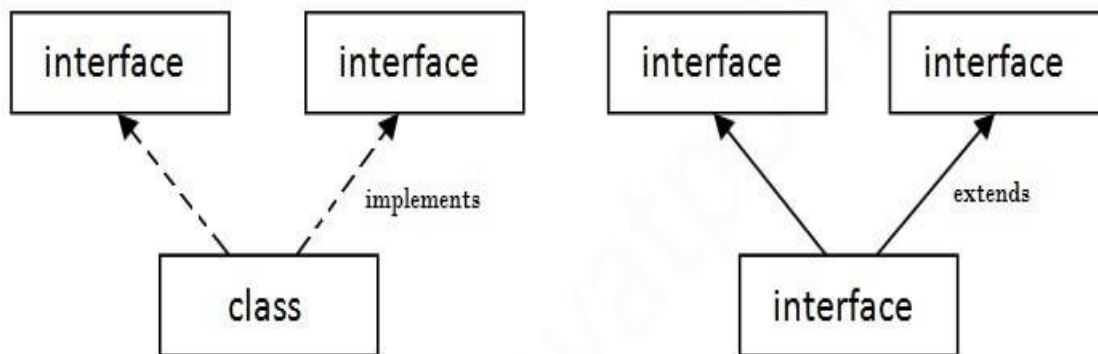
The Java compiler adds **public and abstract** keywords before the interface method. Moreover, it adds public, static and final keywords before data members.

In other words, Interface fields are public, static and final by default, and the methods are public and abstract.



Multiple inheritance in Java by interface

If a class **implements** multiple interfaces, or an interface **extends** multiple interfaces, it is known as **multiple inheritance**.



Multiple Inheritance in Java

```
interface Printable{
    void print();
}
interface Showable{
    void show();
}
class TestInterface implements Printable, Showable{
    public void print(){System.out.println("Hello");}
    public void show(){System.out.println("Welcome");}

    public static void main(String args[]){
        TestInterface obj = new TestInterface ();
        obj.print();
        obj.show();
    }
}
```

Output: Hello
Welcome

Multiple inheritance is not supported through class in java, but it is possible by an interface, why?

As we have explained under inheritance topic, multiple inheritance is not supported in the case of class because of ambiguity. However, it is supported in case of an interface because there is no ambiguity. It is because its implementation is provided by the implementation class. For example:

```
interface Printable{
    void print();
}
interface Showable{
    void print();
}
class TestInterface3 implements Printable, Showable{
    public void print(){System.out.println("Hello");}
    public static void main(String args[]){
        TestInterface3 obj = new TestInterface3();
        obj.print();
    }
}
```

Output:
Hello

As you can see in the above example, Printable and Showable interface have same methods but its implementation is provided by class TestInterface1, so there is no ambiguity.

Interface inheritance

A class implements an interface, but one interface extends another interface.

```
interface Printable{
void print(); }
interface Showable extends Printable{
void show(); }
class TestInterface4 implements Showable{
public void print(){System.out.println("Hello");}
public void show(){System.out.println("Welcome");}

public static void main(String args[]){
TestInterface4 obj = new TestInterface4();
obj.print();
obj.show();
} }
Output:
Hello
Welcome
```

Method Overriding in Java

If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in java**. In other words, if subclass provides the specific implementation of the method that has been provided by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding

- ✓ Method overriding is used to provide specific implementation of a method that is already provided by its super class.
- ✓ Method overriding is used for runtime polymorphism(this will be discussed in the coming section)

Rules for Java Method Overriding

1. The method must have the same name as in the parent class
2. The method must have the same parameter as in the parent class.
3. There must be an IS-A relationship (inheritance).

Let's understand the problem that we may face in the program if we don't use method overriding.

```
class Vehicle{
void run(){System.out.println("Vehicle is running");}
}
class Bike extends Vehicle{

public static void main(String args[]){
Bike obj = new Bike();
obj.run(); } }
```

Chapter 2 Class and objects in Java

Output:- Vehicle is running

The Problem is that we have to provide a specific implementation of **run()** method in subclass that is why we use **method overriding** as in the example below.

Example of method overriding

In this example, we have defined the run method in the subclass as defined in the parent class but it has some specific implementation. The name and parameter of the method is same and there is IS-A relationship between the classes, so there is method overriding.

```
class Vehicle{
void run(){System.out.println("Vehicle is running");}
}
class Bike2 extends Vehicle{
void run(){System.out.println("Bike is running safely");}

public static void main(String args[]){
    Bike2 obj = new Bike2();
    obj.run();
}
```

Difference between method overloading and method overriding in java

There are many differences between method overloading and method overriding in java. A list of differences between method overloading and method overriding are given below:

No.	Method Overloading	Method Overriding
1)	Method overloading is used to increase the readability of the program.	Method overriding is used to provide the specific implementation of the method that is already provided by its super class.
2)	Method overloading is performed within class.	Method overriding occurs in two classes that have IS-A (inheritance) relationship.
3)	In case of method overloading, parameter must be different.	In case of method overriding, parameter must be same.
4)	Method overloading is the example of compile time polymorphism.	Method overriding is the example of run time polymorphism.
5)	In java, method overloading can't be	Return type must be same or covariant in

	performed by changing return type of the method only. <i>Return type can be same or different</i> in method overloading. But you must have to change the parameter.	method overriding.
--	---	--------------------

Polymorphism in java

Polymorphism in Java is a concept by which we can perform a *single action in different ways*. Polymorphism is derived from two Greek words: **poly** and **morphs**. The word "poly" means many and "morphs" means forms. So polymorphism means many forms. There are two types of polymorphism in Java: **compile-time polymorphism** and **runtime polymorphism**. We can perform polymorphism in java by method **overloading** and method **overriding**. The most common use of polymorphism in OOP occurs when a parent class reference is used to refer to a child class object.

If you overload a static method in Java, it is the example of compile time polymorphism. Here, we will focus on runtime polymorphism in java

It is important to know that the only possible way to access an object is through a reference variable. A reference variable can be of only one type. Once declared, the type of a reference variable cannot be changed.

The reference variable can be reassigned to other objects provided that it is not declared final. The type of the reference variable would determine the methods that it can invoke on the object. A reference variable can refer to any object of its declared type or any subtype of its declared type. A reference variable can be declared as a class or interface type.

Virtual Methods:

In this section, we will show you how the behavior of overridden methods in Java allows you to take advantage of polymorphism when designing your classes. We already have discussed method overriding, where a child class can override a method in its parent. An overridden method is essentially hidden in the parent class, and is not invoked unless the child class uses the super keyword within the overriding method.

```
/* File name :Em ployee.java */
public class Employee
{
    private String name;
    private String address;
    private int number;
    public Employee(String name, String address, int number)
    {
        System.out.println("Constructing an Employee");
        this.name = name;
        this.address= address;
    }
}
```

```
this.number= number;
}
public void mailCheck()
{
    System.out.println("Mailing a check to " + this.name
+ " " + this.address);
}
public String toString()
{
    Return name + " " + address + " " + number;
}
public String getName()
{
    return name;
}
public String getAddress()
{
    return address;
}
public void setAddress(String newAddress)
{
    address= newAddress;
}
public int getNumber()
{
    return number;
}}
```

Now suppose we extend Employee class as follows:

/* File name : Salary.java */

```
public class Salary extends Employee
{
    private double salary; //Annual salary
    public Salary(String name, String address, int number, double
salary)
    {
        super(name, address, number);
        setSalary(salary);
    }
    public void mailCheck()
    {
        System.out.println("Within mailCheck of Salary class ");
        System.out.println("Mailing check to " + getName()
```

```
+ " with salary " + salary);
}
public double getSalary()
{
    return salary;
}
public void setSalary(double newSalary)
{
    if(newSalary >= 0.0)
    {
        salary = newSalary;
    }
}
public double computePay()
{
    System.out.println("Computing salary pay for " + getName());
    return salary/52;
}
}
```

Now, you study the following program carefully and try to determine its output:

/ File name :VirtualDemo.java */*

```
public class VirtualDemo
{
    public static void main(String [] args)
    {
        Salary s = new Salary("MohdMohtashim ", "Ambehta, UP", 3, 3600.00);
        Employee e = new Salary("John Adam s", "Boston, MA", 2, 2400.00);
        System.out.println("Call mailCheck using Salary reference --");
        s.mailCheck();
        System.out.println("\n Call mailCheck using Employee reference--");
        e.mailCheck();
    }
}
```

This would produce the following result:

```
Constructing an Employee
Constructing an Employee
Call mailCheck using Salary reference --
Within mailCheck of Salary class
mailing check to MohdMohtashim with salary 3600.0
Call mailCheck using Employee reference--
Within mailCheck of Salary class
mailing check to John Adam s with salary 2400.0
```

Here, we instantiate two *Salary* objects .one using a *Salary* reference *s*, and the other using an *Employee* reference *e*.While invoking *s.mailCheck* the compiler sees **mailCheck** in the *Salary* class at compile time, and the JVM invokes **mailCheck** in the *Salary* class at run time.Invoking *mailCheck* on *e* is quite different because *e* is an *Employee* reference. When the compiler sees *e.mailCheck*, the compiler sees the **mailCheck** method in the *Employee* class.

Here, at compile time, the compiler used **mailCheck** in *Employee* to validate this statement. At runtime, however, the JVM invokes **mailCheck** in the *Salary* class. This behavior is referred to as virtual method invocation, and the methods are referred to as virtual methods. All methods in Java behave in this manner, whereby an overridden method is invoked at run time, no matter what data type the reference is that was used in the source code at compile time

Runtime polymorphism

- ❖ **Runtime polymorphism** or **Dynamic Method Dispatch** is a process in which a call to an overridden method is resolved at runtime rather than compile-time.
- ❖ An overridden method is called through the **reference variable** of a superclass.
- ❖ The determination of the method to be called is based on the object being referred to by the reference variable.
- ❖ Let's first understand the **upcasting** before **Runtime Polymorphism**.

```
class A{}  
class B extends A{  
A a=new B();//upcasting}
```

Example of Java Runtime Polymorphism

- ❖ creating two classes *Bike* and *Splendar*.
- ❖ *Splendar* class extends *Bike* class and overrides its *run()* method.
- ❖ calling the *run* method by the reference variable of Parent class.
- ❖ Since it refers to the subclass object and subclass method overrides the Parent class method, subclass method is invoked at runtime.
- ❖ Since method invocation is determined by the JVM not compiler, it is known as **runtime polymorphism**.

```
class Bike{  
void run()  
{System.out.println("running with 50km");}  
}  
class Splendor extends Bike  
{ void run()  
{ System.out.println("running safely with 60km");  
}  
}  
  
public static void main(String args[])  
{ Bike b = new Splendor();  
//upcasting
```

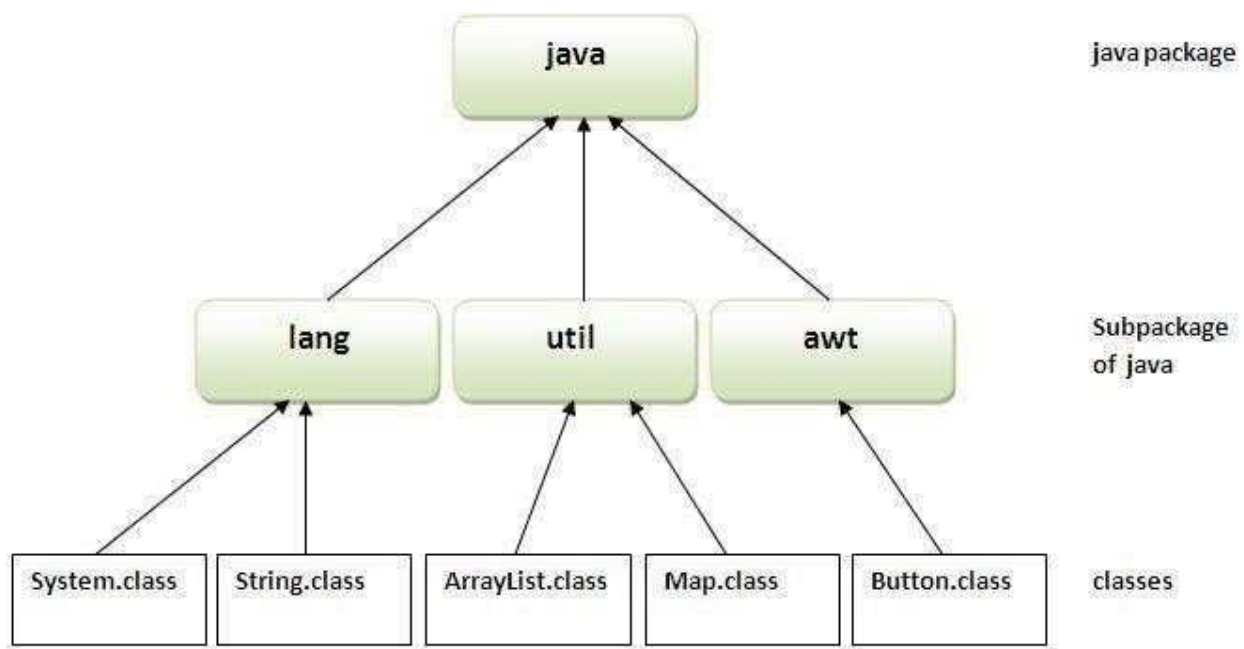
```
b.run(); } }
```

Java packages

Definition: A *package* is a grouping of related types providing access protection and name space management. Note that *types* refer to classes, interfaces. Package in java can be categorized in two form, ***built-in package and user-defined package***. There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc. as shown in diagram below.

Advantage of Java Package

- 1) Java package is used to categorize the classes and interfaces so that they can easily be maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming collision.



The types that are part of the Java platform are members of various packages that bundle classes by function: fundamental classes are in `java.lang`, classes for reading and writing (input and output) are in `java.io`, and so on. You can put your types in packages too.

Creating a Package

To create a package, you choose a name for the package (naming conventions are discussed in the next section) and put a `package` statement with that name at the top of *every source file* that contains the types (classes, interfaces, enumerations, and annotation types) that you want to include in the package.

```
//save as Simple.java
```

```
package mypack;
public class Simple{
    public static void main(String args[]){
        System.out.println("Welcome to package");
    } }
////////////////////////////////////
//save by A.java
package pack;
public class A{
    public void msg(){System.out.println("Hello");}
}
////////////////////////////////////
//save by B.java
package mypack;
import pack.*;
class B{
    public static void main(String args[]){
        A obj = new A();
        obj.msg();
    }
}
```

How to access package from another package?

There are three ways to access the package from outside the package.

1. import package.*;
2. import package.classname;
3. fully qualified name.

*1) Using packagename.**

If you use package.* then all the classes and interfaces of this package will be accessible but not subpackages.

The import keyword is used to make the classes and interface of another package accessible to the current package.

*Example of package that import the packagename.**

```
//save by A.java
package pack;
public class A{
    public void msg(){System.out.println("Hello");}
}
//save by B.java
```

```
package mypack;
import pack.*;

class B{
    public static void main(String args[]){
        A obj = new A();
        obj.msg();
    }
}
```

2) Using *packagename.classname*

If you import *package.classname* then only declared class of this package will be accessible.

Example of package by import package.classname

```
//save by A.java

package pack;
public class A{
    public void msg(){System.out.println("Hello");}
}

//save by B.java
package mypack;
import pack.A;

class B{
    public static void main(String args[]){
        A obj = new A();
        obj.msg();
    }
}
Output:Hello
```

3) Using *fully qualified name*

If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface. It is generally used when two packages have same class name e.g. *java.util* and *java.sql* packages contain Date class.

Example of package by import fully qualified name

```
//save by A.java
package pack;
public class A{
```

```
public void msg(){System.out.println("Hello");}
}
//save by B.java
package mypack;
class B{
  public static void main(String args[]){
    pack.A obj = new pack.A();//using fully qualified name
    obj.msg();
  } }
```

Output: Hello

If you import a package, all the classes and interface of that package will be imported excluding the classes and interfaces of the subpackages. Hence, you need to import the subpackage as well.

Sub-package in java

Package inside the package is called the **subpackage**. It should be created **to categorize the package further**

Naming a Package

With programmers worldwide writing classes and interfaces using the Java programming language, it is likely that many programmers will use the same name for different types.

Naming Conventions

Package names are written in all lower case to avoid conflict with the names of class or interface.